

• ARCHITECTURE & CHANGE REPORT

Firefly Security Platform

fireflyframework-security

Converting **fireflyframework-idp** into a SOTA, **product-agnostic**, hexagonal security platform on **Spring Security 6** — secure-by-default, delivered **with and without the starters**.

14 modules

hexagonal · ports & adapters

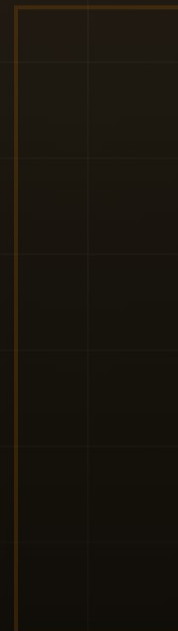
secure-by-default

5 real-Docker ITs

Contents

Architecture · Change Report

Part I — The Platform.....	3
1. Executive summary.....	4
2. What changed — at a glance.....	6
3. Secure-by-default & product-agnostic.....	7
Part II — Architecture.....	9
4. The hexagonal model.....	10
5. With and without the starters.....	12
6. Request flow & port coverage.....	13
Part III — Delivery & Change Log.....	15
7. Repositories, branches & PRs.....	16
8. Verification & test evidence.....	18
9. What remains & how to consume.....	19
Appendix A · Figures.....	20



PART I

The Platform

Why **fireflyframework-idp** became **fireflyframework-security** — the secure-by-default posture and the product-agnostic domain.

-
- 01 Executive summary
 - 02 What changed — at a glance
 - 03 Secure-by-default & product-agnostic
-

PART I · THE PLATFORM · Chapter 1

Executive summary

The Firefly framework now has a SOTA, **product-agnostic**, hexagonal security platform — `fireflyframework-security` — built on **Spring Security 6** and delivered, secure-by-default, through the application starter.

Before this initiative, Firefly's security was **two disconnected halves**. One was an identity broker (`fireflyframework-idp` plus four adapters) exposing a single fat `IdpAdapter` port. The other was a bespoke, AOP-based authorization scheme buried in the application starter. The two never met: obtaining a token did nothing to populate the context the aspect checked, **no Spring Security was on the classpath**, inbound JWT signatures were never validated, identity entered through a *trusted X-Party-Id* header, and the default posture was fail-**open**.

The platform closes every one of those gaps. It introduces a clean hexagon — driving ports and a product-agnostic domain, driven SPI ports, a vendor-neutral engine, a reactive Spring Security binding, delivery modules, and pluggable adapters — joined to the IdP tier by exactly **one seam**, `TokenValidationPort`. It is **secure-by-default**, **hard**, and it has been verified against **real infrastructure** (Postgres, OPA, Cerbos, OpenFGA and Vault containers).

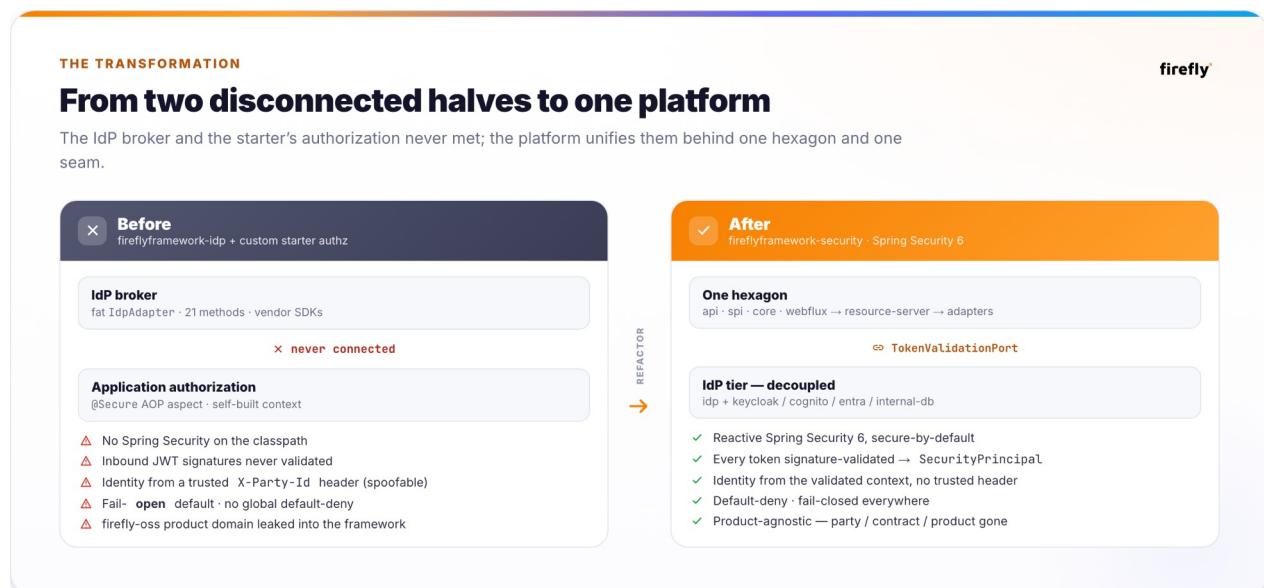


Figure 1. The transformation: two disconnected halves — an IdP broker and a fail-open AOP authorization layer — become one secure-by-default hexagonal platform joined by a single seam.

♦ The headline

Identity is now always a **signature-validated** `SecurityPrincipal`, never a trusted header. Every route is default-deny unless explicitly permitted. Every authorization decision is fail-closed. And the framework carries **no product domain** — party, contract and product are gone.

By the numbers	Result
New modules	14 — all build green and published
Main source	~3,100 lines of Java across the security modules
Test suites	21, incl. 8 integration tests
Real-container integration tests	5 (Postgres · OPA · OpenFGA · Vault · Cerbos)
SPI port coverage	every driven port has at least one real adapter
Repositories affected	14 new + 7 modified (with open PRs)
Releases	none — code pushed; no GitHub/Maven release

PART I · THE PLATFORM · Chapter 2

What changed — at a glance

The work spans new modules, a refactor of the IdP tier, structural changes to the build, and a framework-wide de-domain. The table summarizes each area; later chapters expand them.

Area	Change
Security platform	Created 14 modules: the hexagon (api/spi/core/webflux), delivery (resource-server, method-policy, oauth2-client, authorization-server), adapters (opa, cerbos, openfga, vault, r2dbc) and test fixtures.
IdP tier	Segregated the fat IdpAdapter into six capability ports with NotSupported defaults (an additive change — all four adapters still build); de-domained its DTOs.
Starters	Wired the secure-by-default resource server + method security into fireflyframework-starter-application ; only core / domain / application starters remain.
BOM & parent	Consolidated to a single fireflyframework-bom ; added observability + the security platform to fireflyframework-parent 's dependency management.
Aggregator	A root reactor module was created, then removed (repo deleted) in favour of the parent + BOM approach.
De-domain	Removed X-Party-Id , party / contract / product and the Security Center SPI from the application and backoffice layers; identity now comes from the validated security context.
Skills	Added two Claude-Code skills and a security agent to fireflyframework-claude-skills .

The platform was shaped by a set of explicit decisions: **secure-by-default**, **hard** (no fail-open); **product-agnostic**, with a clean break and no compatibility shims; **reactive-only** except the servlet authorization server; segregated IdP ports; an embedded policy engine plus **OPA**, **Cerbos** and **OpenFGA** adapters; **Vault** for secrets; and **R2DBC** (not Redis) for data-backed repositories.

PART I · THE PLATFORM · Chapter 3

Secure-by-default & product-agnostic

Secure-by-default, hard

The resource server ships an opinionated posture that an application opts *out* of, never *into*:

- **Default-deny** — every route requires a validated bearer token unless it is in an explicit **permit-matchers** list.
- **In-process validation** — JWT signatures are verified against JWKS with issuer / audience / expiry validators; opaque tokens go through RFC 7662 introspection with a bounded cache.
- **Fail-closed** — a policy or introspection error denies; method security throws; nothing permits on error.
- **Hardened headers** — HSTS, CSP, frame-options and Referrer-Policy on every response; CSRF off for pure bearer APIs.
- No fail-open switch — the legacy **security.enforce**-style bypass is gone.

▲ Breaking on upgrade (by design)

Because the posture flips to default-deny and real validation, services that previously relied on fail-open behaviour or the trusted **X-Party-Id** header will start returning **401/403** until they are configured. This is the intended outcome of closing the security holes — a clean break, no shims.

Product-agnostic

The framework must carry no concepts from any product family. The initiative purged the firefly-oss domain (**X-Party-Id**, **partyId**, business-role enums, and a contract / product “Security Center” session model) from the IdP, application and backoffice layers, replacing them with generic primitives. Products re-introduce their own model through **SecurityPrincipal.attributes** and a contributor port.

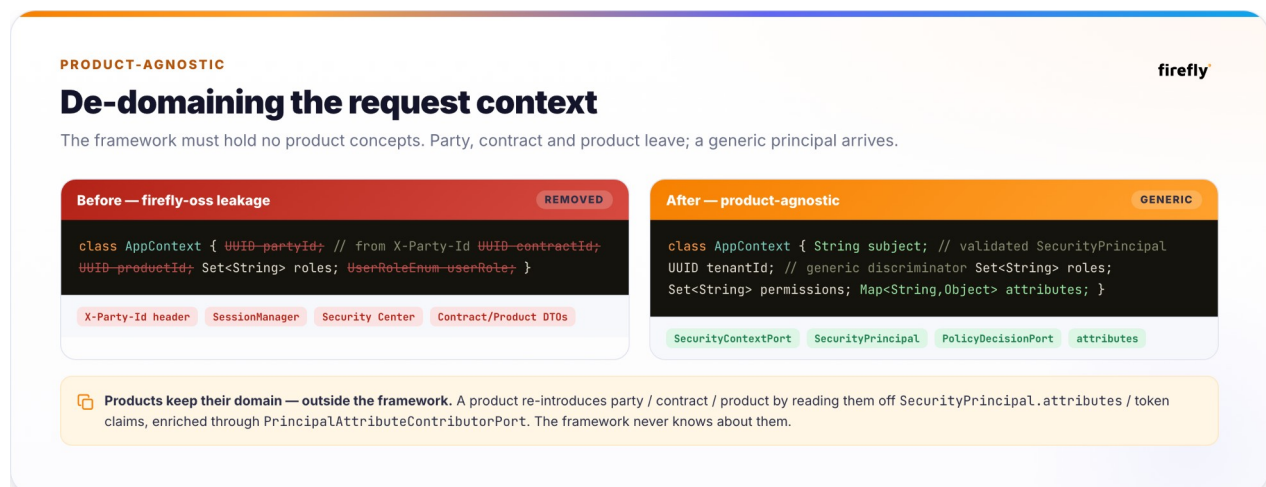


Figure 2. De-domaining the request context: the firefly-oss party/contract/product fields and the trusted ‘X-Party-Id’ header give way to a generic subject, tenant and attributes projected from the validated principal.

Removed (product leakage)	Replaced with (generic)
X-Party-Id trusted header	identity from the validated <code>SecurityContextPort</code> / <code>SecurityPrincipal</code>
partyId (UUID)	subject (String) from the token
contractId / productId scoping	generic <code>tenantId</code> + open <code>attributes</code> map
UserRoleEnum business roles	plain <code>Set<String></code> authorities
Security Center <code>SessionManager</code> + DTOs	<code>PolicyDecisionPort</code> + <code>AuthorityMappingPort</code> (product re-implements its own)



PART II

Architecture

The hexagonal port/adaptor model, delivery **with and without the starters**, the request flow, and full SPI → adaptor coverage.

-
- 04 The hexagonal model
 - 05 With & without the starters
 - 06 Request flow & port coverage
-

PART II · ARCHITECTURE · Chapter 4

The hexagonal model

Driving ports and a product-agnostic domain at the top; driven ports and pluggable adapters at the bottom; a vendor-neutral engine in the middle. The core never imports a provider SDK.

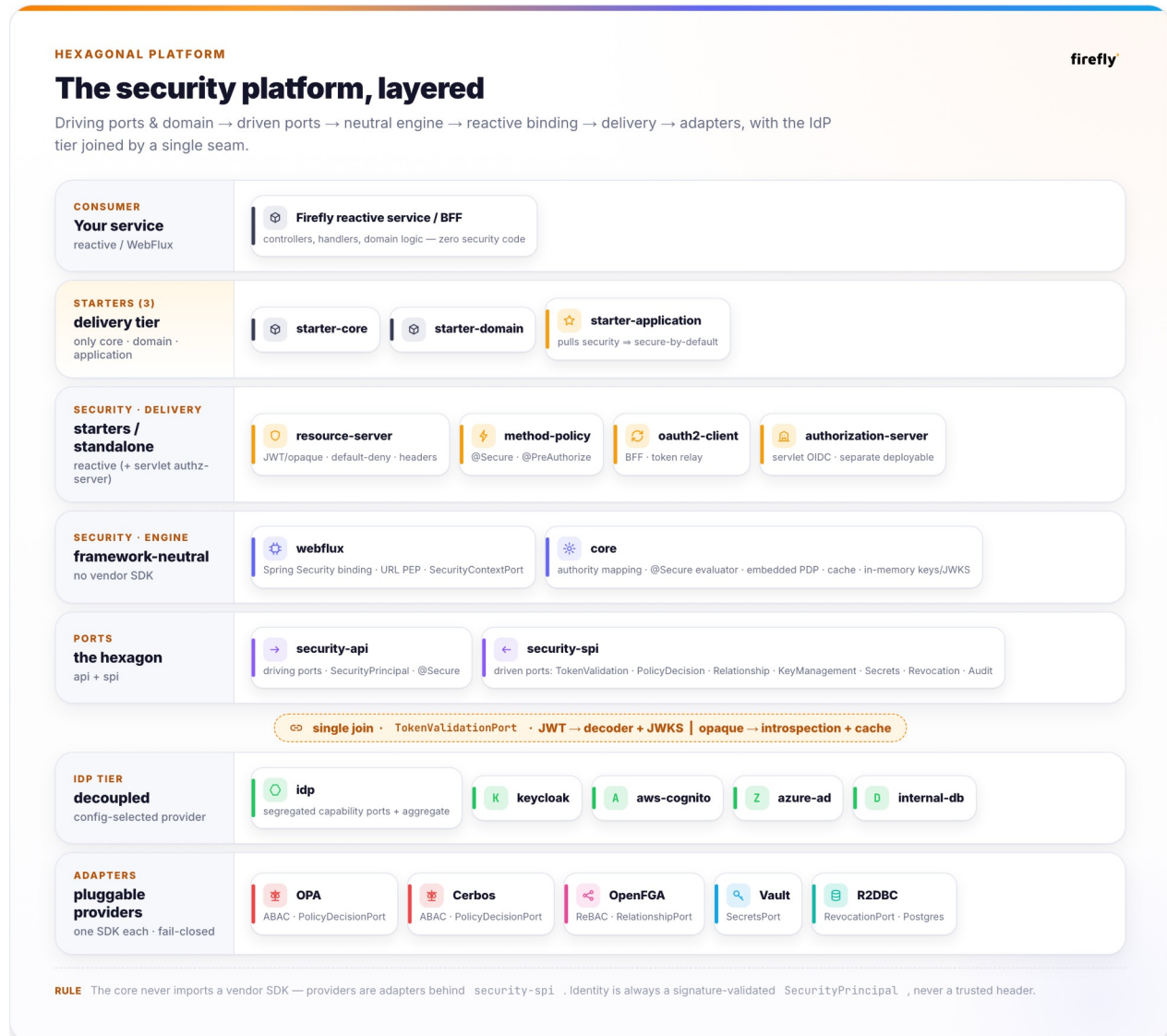


Figure 3. The security platform in layers — from the consumer service down to the pluggable adapters, with the IdP tier joined by the single `TokenValidationPort` seam.

Each band is a Maven module (or group). `security-api` defines the driving ports, the `@Secure` annotation and the `SecurityPrincipal`. `security-spi` declares the driven ports. `security-core` is the framework-neutral engine: authority mapping, the `@Secure` evaluator, the embedded fail-closed policy engine, the introspection cache and the in-memory key/JWKS source. `security-webflux` binds it to Spring Security; the delivery modules expose it; the adapters provide concrete backends.

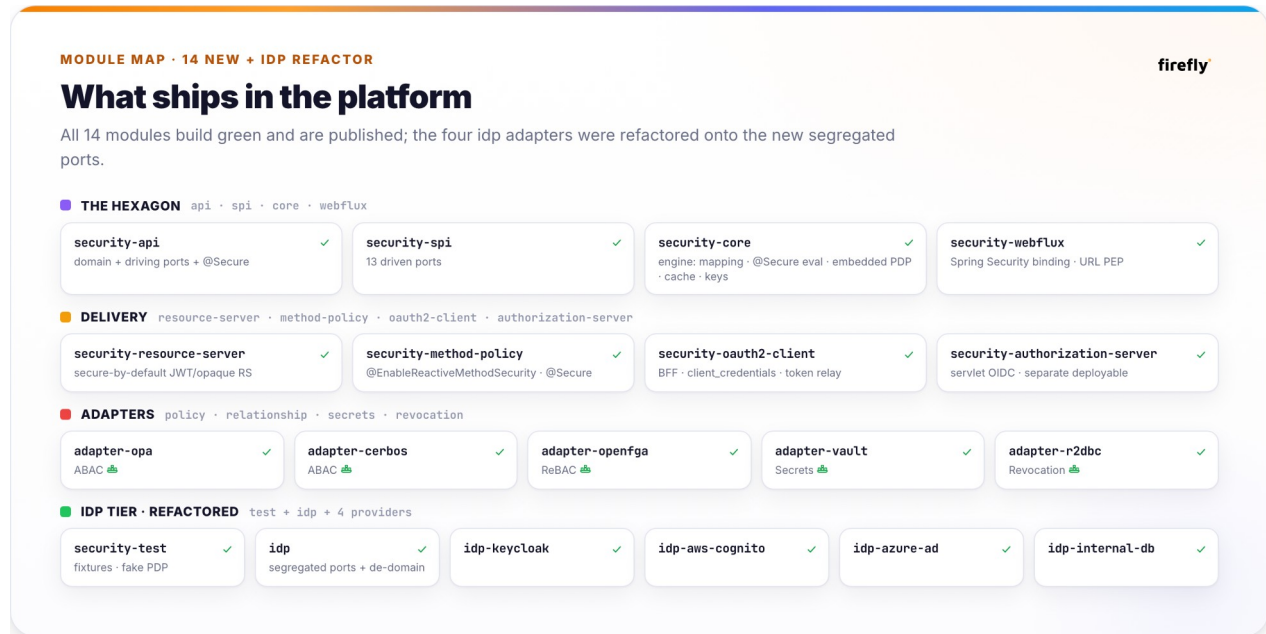


Figure 4. The fourteen new modules plus the refactored IDP tier — all building green and published to GitHub.

PART II · ARCHITECTURE · Chapter 5

With and without the starters

The same secure-by-default resource server reaches a service two ways: transitively through the application starter, or added directly as a standalone dependency.

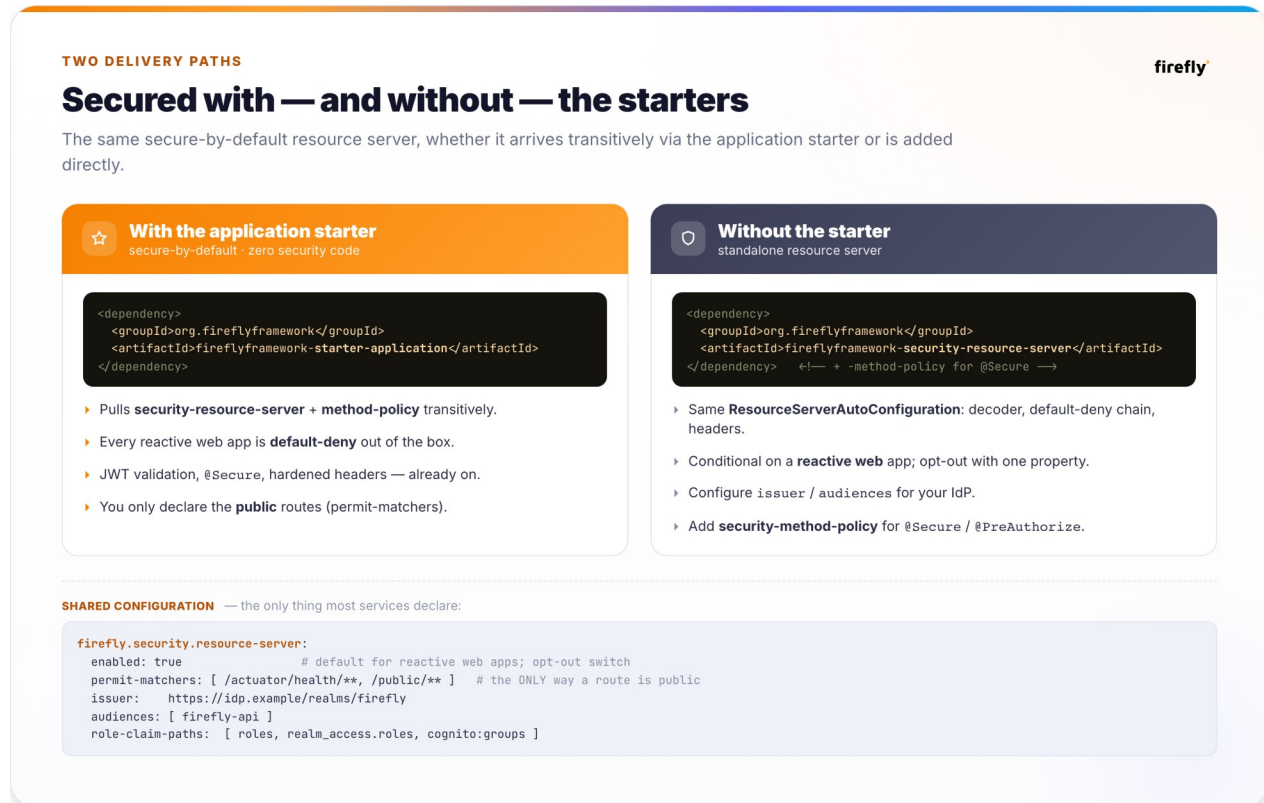


Figure 5. Two delivery paths to the same outcome — secure-by-default via the application starter, or a standalone resource server.

With the application starter

fireflyframework-starter-application depends on **security-resource-server** and **security-method-policy**. Any application-tier service is locked down out of the box: default-deny, JWT validation, hardened headers and **@Secure** method security, with **zero security code**. The service only declares which routes are public.

Without the starter

A service that is not on the application starter adds **fireflyframework-security-resource-server** directly (and **-method-policy** for **@Secure**). It receives the identical **ResourceServerAutoConfiguration**. The autoconfiguration is conditional on a reactive web application and can be disabled with a single property, so it composes cleanly with any service.

PART II · ARCHITECTURE · Chapter 6

Request flow & port coverage

A request is validated, projected into a principal, authorized at the URL and method layers, and audited — all fail-closed.

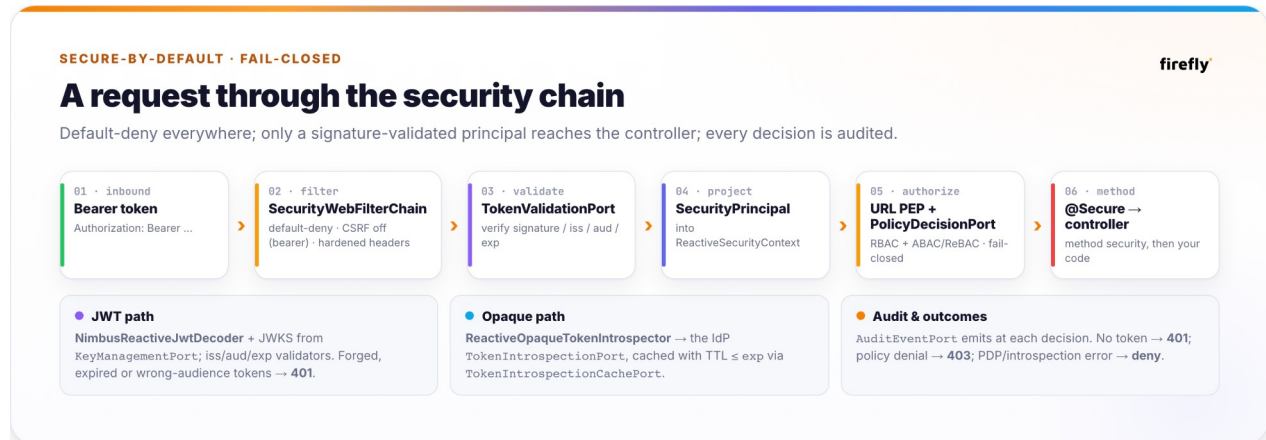


Figure 6. A request through the security chain: validate → project → authorize → method security → controller, with JWT and opaque branches and audit at every decision.

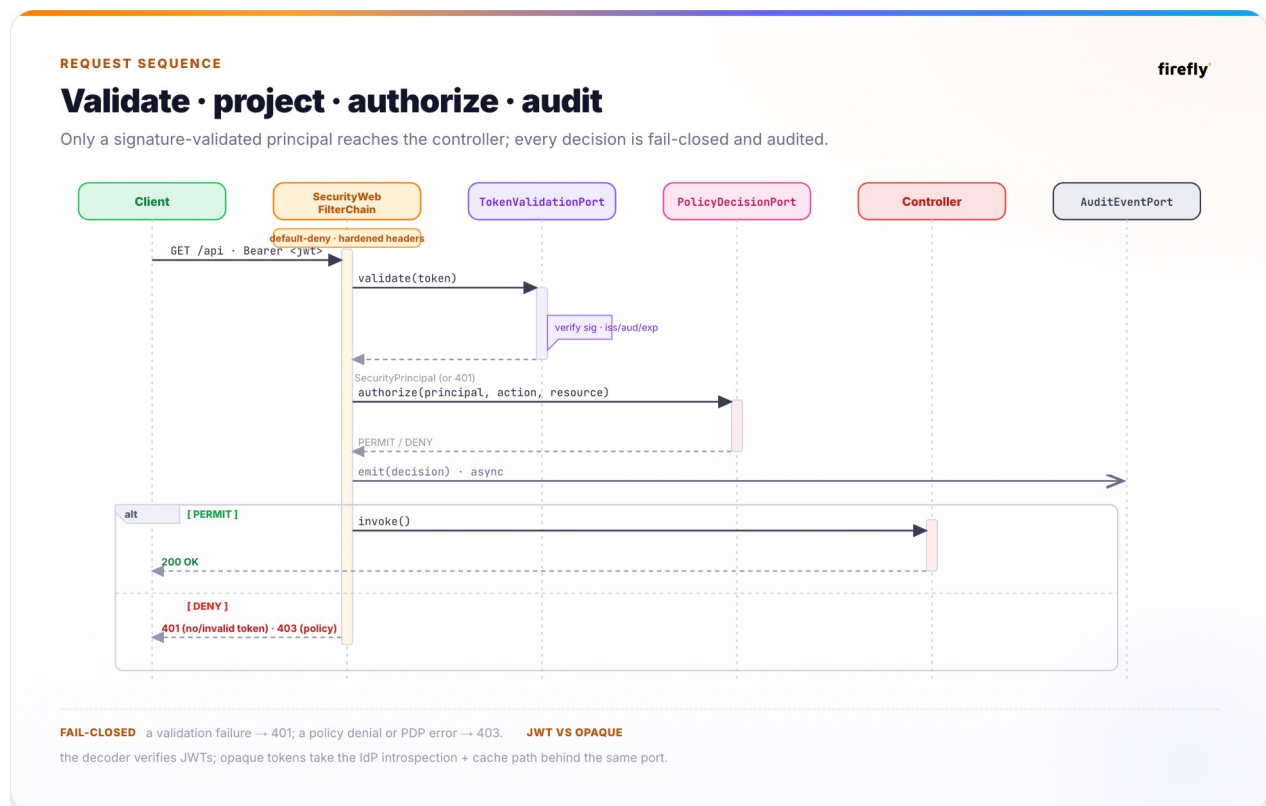


Figure 7. The validation-and-authorization sequence: the bearer token is validated, projected into a principal, authorized at the URL and method layers, and audited — with explicit permit and deny outcomes.

Authorization composes RBAC (authorities/scopes at the URL and method layers) with optional ABAC and ReBAC through the **PolicyDecisionPort** and **RelationshipPort**. The default policy engine is embedded and fail-closed; production swaps in an OPA, Cerbos or OpenFGA adapter by declaring a bean.

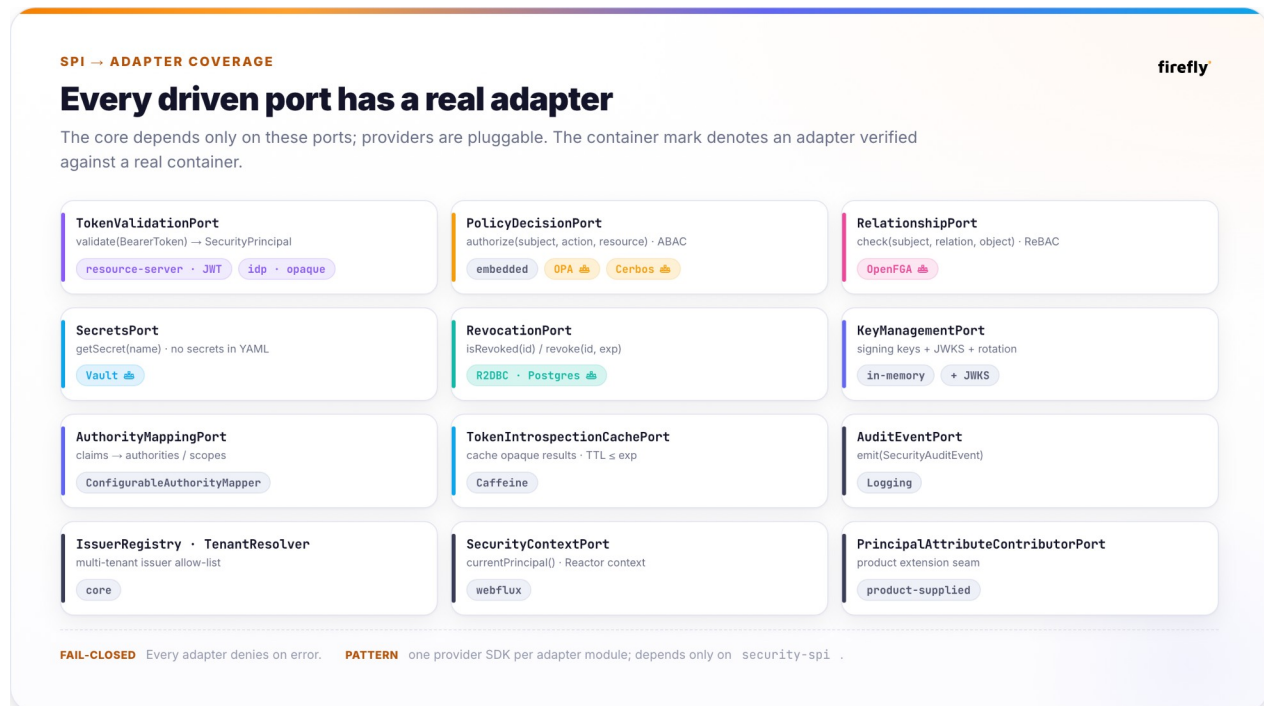


Figure 8. SPI → adapter coverage. Every driven port has at least one real implementation; the container icon marks adapters verified against a live service.

i One seam, cleanly

The IdP tier (fireflyframework-idp + the Keycloak / Cognito / Entra / internal-db providers) stays decoupled from the security tier. They meet only at **TokenValidationPort** — the same shape the Python (pyfly) and Rust ports already validated.



PART III

Delivery & Change Log

Every repository, branch, pull request and status — and how the platform was verified against **real infrastructure**.

07 Repositories, branches & PRs

08 Verification & test evidence

09 What remains & how to consume

PART III · DELIVERY & CHANGE LOG · Chapter 7

Repositories, branches & PRs

Fourteen new public repositories under `github.com/fireflyframework`, each with a README, description and curated topics; seven existing repositories changed behind open, unmerged PRs.

New repositories (14) — pushed to `main`

Repository (fireflyframework-...)	Role
security-api	driving ports + domain + @Secure
security-spi	13 driven ports
security-core	neutral engine (mapping, @Secure eval, embedded PDP, cache, keys)
security-webflux	reactive Spring Security binding + URL PEP
security-resource-server	secure-by-default JWT/opaque resource server
security-method-policy	@EnableReactiveMethodSecurity + @Secure interceptor
security-oauth2-client	reactive OAuth2/OIDC client (BFF)
security-authorization-server	servlet OIDC authorization server
security-adapter-opa	ABAC PolicyDecisionPort (OPA)
security-adapter-cerbos	ABAC PolicyDecisionPort (Cerbos)
security-adapter-openfga	ReBAC RelationshipPort (OpenFGA)
security-adapter-vault	SecretsPort (HashiCorp Vault)
security-adapter-r2dbc	RevocationPort (relational DB)
security-test	test fixtures + fake PDP

Modified repositories — open pull requests (do not merge)

Repository	PR	Branch	Change
fireflyframework-idp	#13	feature/agnostic-de-domain	segregate ports + de-domain DTOs
fireflyframework-idp-keycloak	#13	feature/agnostic-de-domain	drop partyId/userRole writes

Repository	PR	Branch	Change
fireflyframework-bom	#22	feature/security-platform-bom	manage the 14 security modules
fireflyframework-parent	#30	feature/manage-observability-and-security	observability + security in dep-mgmt
fireflyframework-starter-application	#12	feature/wire-security-platform	wire security + de-domain (180 tests)
fireflyframework-backoffice	#11	feature/agnostic-de-domain	de-domain (13 tests)
fireflyframework-claude-skills	#1	feature/security-skills	security skills + agent (v1.21.0)

Aggregator removed

A **fireflyframework-aggregator** reactor module was briefly created and pushed, then **deleted** (local module and GitHub repository) once the decision was to manage modules through the parent and BOM instead.

PART III · DELIVERY & CHANGE LOG · Chapter 8

Verification & test evidence

The platform is not asserted to work — it is shown to. Beyond unit tests, eight integration tests exercise real cryptography and real services; five spin up genuine Docker containers via Testcontainers.

What is verified	Against	Result
Resource server: signed JWT → 200; no/forged/expired → 401; policy deny → 403; headers	in-process RS256 crypto	7 ✓
Token revocation (RevocationPort)	real PostgreSQL container	4 ✓
ABAC policy (PolicyDecisionPort)	real OPA container	4 ✓
ABAC policy (PolicyDecisionPort)	real Cerbos container	2 ✓
ReBAC relationships (RelationshipPort)	real OpenFGA container	2 ✓
Secrets (SecretsPort)	real HashiCorp Vault container	3 ✓
Authorization server: OIDC discovery, JWKS, client_credentials	servlet @SpringBootTest	3 ✓
OAuth2 client: client_credentials exchange	WireMock token endpoint	1 ✓
Method security: @Secure / @PreAuthorize	reactive security context	5 ✓
Application layer (de-domained)	full module test suite	180 ✓
Backoffice layer (de-domained)	full module test suite	13 ✓

✓ Green end-to-end

All 14 security modules install together with zero failures; the segregated IdP and the de-domained application and backoffice layers build green. Every claim in this report is backed by a passing build.

What remains & how to consume

What remains

- **Cloud key/secret adapters** (AWS KMS, Azure Key Vault) — the in-memory **KeyManagementPort** and the Vault **SecretsPort** cover today's needs; these are provider variants that cannot be verified against a local container.
- **Deeper per-adapter hardening** of the legacy IdP adapters (offloading blocking SDK calls, replacing any remaining local token parsing with verifying decoders) — a focused quality pass.
- **Merging the pull requests** — all seven PRs are intentionally left open for review; nothing is released to GitHub or Maven yet.

How to consume

1. On the application starter? You are already secure-by-default — declare your public routes in `firefly.security.resource-server.permit-matchers`.
2. Standalone? Add `fireflyframework-security-resource-server` (and `-method-policy` for `@Secure`); set `issuer` / `audiences` and your provider's claim paths.
3. Annotate service methods with `@Secure`; never trust a header for identity.
4. Need ABAC / ReBAC / secrets / revocation? add the OPA, Cerbos, OpenFGA, Vault or R2DBC adapter and declare the port bean.

♦ The bottom line

`fireflyframework-idp` became `fireflyframework-security`: a hexagonal, product-agnostic, secure-by-default platform on Spring Security 6 — built, tested against real infrastructure, documented, and delivered through the starters and without them.

APPENDICES

Appendix A • Figures

Every figure in this document, in reading order.

Fig.	Title
1	The transformation
2	De-domaining the request context
3	The security platform in layers — from the consumer service down...
4	The fourteen new modules plus the refactored IdP tier — all built...
5	Two delivery paths to the same outcome — secure-by-default via t...
6	A request through the security chain
7	The validation-and-authorization sequence
8	SPI → adapter coverage